

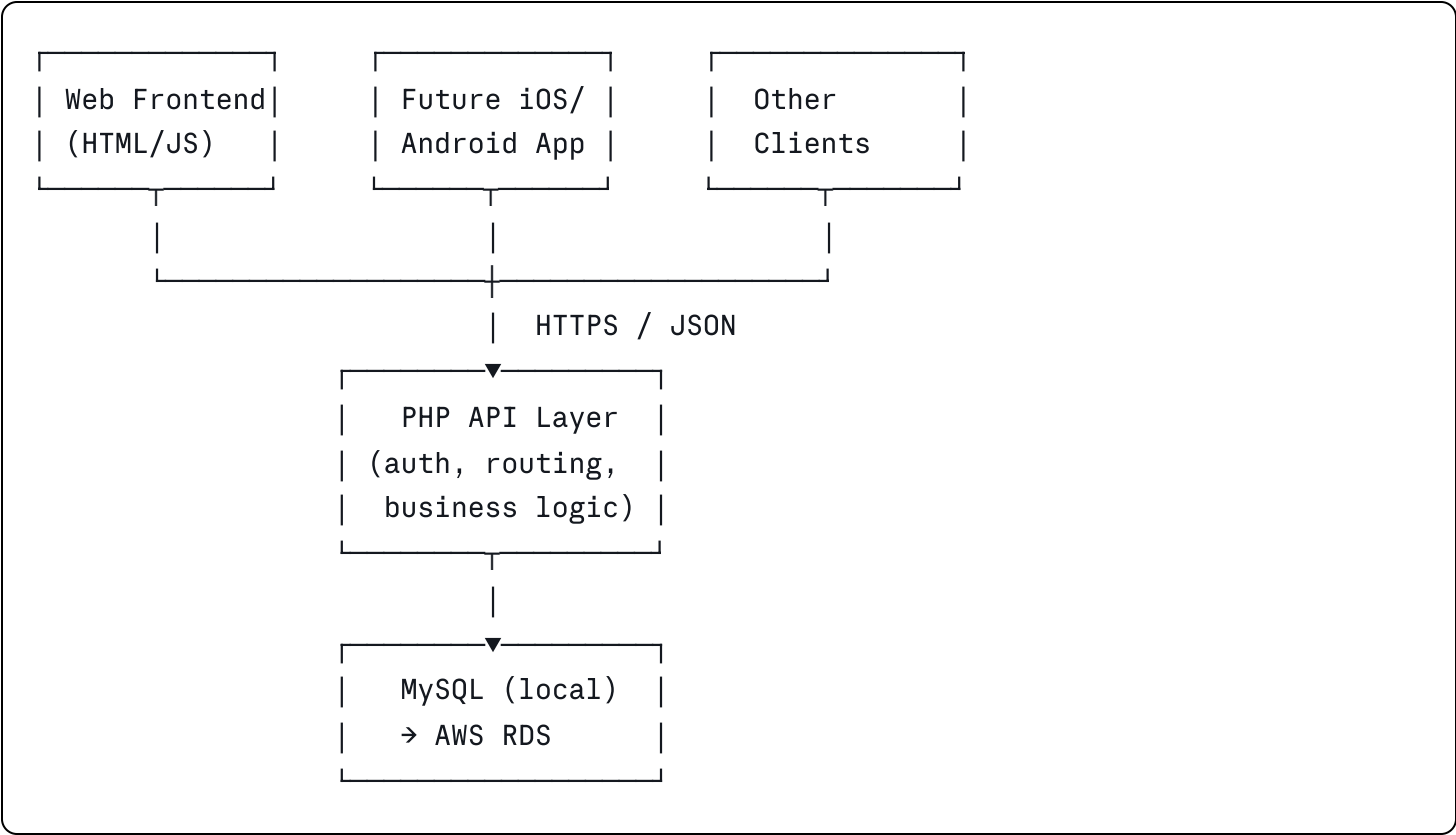
CareerScore — Architecture & Schema Design Document

Stack: PHP (API-first backend) · MySQL (local → AWS RDS) · Responsive HTML/CSS/JS frontend
Version: 0.1 (foundation)

1. System Architecture

1.1 API-First Approach

Because a native mobile app will eventually consume the same backend, PHP is structured as a **JSON API layer** rather than page-rendering scripts. The web frontend and the future app are both *clients* of this API.



1.2 Request Flow

1. Client sends request to `/api/...` with a session token (JWT or PHP session) in the header.
2. A central router (`index.php` front controller) dispatches to the correct endpoint.
3. Endpoint validates auth, validates input, calls business logic, queries MySQL.
4. Response returned as JSON with a consistent envelope: `{ "success": bool, "data": {...}, "error": null }`.

1.3 Local → AWS Migration Path

- **Local:** PHP built-in server or XAMPP/MAMP, local MySQL (e.g. MariaDB).
 - **AWS target:** EC2 or Elastic Beanstalk for PHP, **RDS for MySQL**, S3 for profile pictures, SES for email verification.
 - Keep DB credentials and storage paths in a single `config.php` / `.env` so the switch is a config change, not a code change. Store images by reference (path/URL) so moving from local disk → S3 only changes the storage adapter.
-

2. Project Folder Structure

```
careerscore/
├── public/                # web root (only this is exposed)
│   ├── index.php         # front controller / router
│   ├── assets/
│   │   ├── css/
│   │   ├── js/
│   │   └── img/
│   └── uploads/          # profile pics (local; → S3 later)
├── api/
│   ├── auth/             # register, login, logout, verify
│   ├── profile/          # profile CRUD, attributes
│   ├── feed/             # personal + main feed
│   ├── follow/           # follow/unfollow
│   ├── score/            # Score Me! endpoints
│   └── company/          # company signup & profiles
├── src/
│   ├── Database.php      # PDO connection (single source)
│   ├── Auth.php          # session/token + password logic
│   ├── Response.php      # JSON envelope helper
│   └── models/           # User, Company, Post, Score, etc.
├── config/
│   ├── config.php        # env-driven settings
│   └── schema.sql        # full DB schema (section 3)
└── migrations/           # incremental schema changes
```

Security note: only `public/` is the web root. Everything else sits above it, unreachable by direct URL.

3. MySQL Schema

Design goals: (a) core/stable data as columns, (b) **flexible attributes** so new profile fields need no migrations, (c) repeating structured data normalized into child tables.

All tables: `InnoDB`, `utf8mb4`. Timestamps via `created_at` / `updated_at`.

3.1 Accounts: `users`

Core, stable fields only.

```
sql

CREATE TABLE users (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  uuid        CHAR(36) NOT NULL UNIQUE,          -- public-facing ID
  email       VARCHAR(255) NOT NULL UNIQUE,      -- one account per email
  username    VARCHAR(50)  NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,          -- password_hash()/bcrypt
  city        VARCHAR(100),
  state       VARCHAR(100),
  country     VARCHAR(100),
  profile_pic  VARCHAR(255),                    -- path or S3 URL
  is_verified  TINYINT(1) NOT NULL DEFAULT 0,   -- email verification
  is_active    TINYINT(1) NOT NULL DEFAULT 1,
  created_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
              ON UPDATE CURRENT_TIMESTAMP,
  INDEX idx_location (country, state, city)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

The `UNIQUE` constraint on `email` enforces one-account-per-email at the DB level — the strongest guarantee.

3.2 Companies / Organizations: `companies`

Companies sign up as their own entity and can be followed.

```
sql
```

```
CREATE TABLE companies (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  uuid        CHAR(36) NOT NULL UNIQUE,
  email       VARCHAR(255) NOT NULL UNIQUE,
  name        VARCHAR(150) NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  industry    VARCHAR(100),
  city        VARCHAR(100),
  state       VARCHAR(100),
  country     VARCHAR(100),
  logo        VARCHAR(255),
  website     VARCHAR(255),
  description  TEXT,
  is_verified TINYINT(1) NOT NULL DEFAULT 0,
  is_active   TINYINT(1) NOT NULL DEFAULT 1,
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
              ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

3.3 Flexible Profile Attributes: `user_profile_attributes`

This is the table that solves "add new fields easily." New profile fields you invent later need no

`ALTER TABLE` — you just insert a new attribute key.

```
sql

CREATE TABLE user_profile_attributes (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id     BIGINT UNSIGNED NOT NULL,
  attr_key    VARCHAR(64) NOT NULL,      -- e.g. 'headline', 'bio', 'career_goal'
  attr_value  TEXT,
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
              ON UPDATE CURRENT_TIMESTAMP,
  UNIQUE KEY uniq_user_attr (user_id, attr_key),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  INDEX idx_attr_key (attr_key)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Optional companion: an `attribute_definitions` table can describe each key (label, input type, validation rules, whether it's public). That lets the frontend render new fields

dynamically and lets you add fields by inserting a row — no code change. Recommended once the field list grows.

```
sql

CREATE TABLE attribute_definitions (
  attr_key    VARCHAR(64) PRIMARY KEY,
  label       VARCHAR(120) NOT NULL,
  input_type  ENUM('text', 'textarea', 'number', 'date', 'select', 'bool') NOT NULL,
  options     JSON,           -- for select types
  is_public   TINYINT(1) NOT NULL DEFAULT 1,
  sort_order  INT NOT NULL DEFAULT 0
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Tradeoff to know: key-value attributes are flexible but slower to query/filter in bulk. Anything you'll *score on or filter heavily* (skills, job titles) gets its own proper table below. Use the flexible table for descriptive, display-only fields.

3.4 Structured Repeating Data

Each user has many of these, so they're normalized child tables.

Job history

```
sql

CREATE TABLE job_history (
  id            BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id       BIGINT UNSIGNED NOT NULL,
  title         VARCHAR(150) NOT NULL,
  company_name  VARCHAR(150),
  company_id    BIGINT UNSIGNED NULL,      -- linked if it's a registered company
  start_date    DATE,
  end_date      DATE,                     -- NULL = current
  description   TEXT,
  created_at    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (company_id) REFERENCES companies(id) ON DELETE SET NULL,
  INDEX idx_user (user_id),
  INDEX idx_title (title)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Education / degrees

```
sql
```

```
CREATE TABLE education (  
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  user_id     BIGINT UNSIGNED NOT NULL,  
  institution VARCHAR(150),  
  degree      VARCHAR(150),  
  field       VARCHAR(150),  
  start_year  SMALLINT,  
  end_year    SMALLINT,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
  INDEX idx_user (user_id)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Certifications

```
sql  
  
CREATE TABLE certifications (  
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  user_id     BIGINT UNSIGNED NOT NULL,  
  name        VARCHAR(150) NOT NULL,  
  issuer      VARCHAR(150),  
  issue_date  DATE,  
  expiry_date DATE,  
  credential_id VARCHAR(100),  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
  INDEX idx_user (user_id),  
  INDEX idx_name (name)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Skills — normalized with a master list so scoring/filtering is fast and consistent.

```
sql
```

```

CREATE TABLE skills (
  id      BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name    VARCHAR(100) NOT NULL UNIQUE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE user_skills (
  user_id      BIGINT UNSIGNED NOT NULL,
  skill_id     BIGINT UNSIGNED NOT NULL,
  proficiency  TINYINT,           -- optional 1-5 self-rating
  PRIMARY KEY (user_id, skill_id),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (skill_id) REFERENCES skills(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Career interests — same pattern (master list + join), powers feed and scoring.

```

sql

CREATE TABLE interests (
  id      BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name    VARCHAR(100) NOT NULL UNIQUE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE user_interests (
  user_id      BIGINT UNSIGNED NOT NULL,
  interest_id  BIGINT UNSIGNED NOT NULL,
  PRIMARY KEY (user_id, interest_id),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (interest_id) REFERENCES interests(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

3.5 Following / Linking: **follows**

A user can follow other users **or** companies. A single polymorphic table handles both; following can be globally disabled per user via settings (section 3.8).

```

sql

```

```
CREATE TABLE follows (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  follower_id BIGINT UNSIGNED NOT NULL,          -- always a user
  target_type ENUM('user','company') NOT NULL,
  target_id   BIGINT UNSIGNED NOT NULL,
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uniq_follow (follower_id, target_type, target_id),
  FOREIGN KEY (follower_id) REFERENCES users(id) ON DELETE CASCADE,
  INDEX idx_target (target_type, target_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Note: `target_id` is polymorphic, so it can't use a single FK constraint. Integrity is enforced in application logic. (If you prefer strict DB-level FKs later, split into `user_follows` + `company_follows`.)

3.6 Feeds: `posts` and `feed_items`

`posts` — content authored by a user or company (powers the personal feed page).

```
sql

CREATE TABLE posts (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  author_type ENUM('user','company') NOT NULL,
  author_id   BIGINT UNSIGNED NOT NULL,
  body        TEXT NOT NULL,
  media_url   VARCHAR(255),
  visibility  ENUM('public','followers') NOT NULL DEFAULT 'public',
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
              ON UPDATE CURRENT_TIMESTAMP,
  INDEX idx_author (author_type, author_id),
  INDEX idx_created (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

`feed_items` — the main feed. Your feed algorithm is undecided, so this table just records *what could appear in whose feed and why*, which any future algorithm can rank. (Start by generating these from follows; evolve the ranking later.)

```
sql
```



```
CREATE TABLE feed_items (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id     BIGINT UNSIGNED NOT NULL,      -- whose feed this item is for
  post_id     BIGINT UNSIGNED NOT NULL,
  reason      VARCHAR(64),                  -- 'followed','interest_match', etc.
  score       FLOAT DEFAULT 0,              -- ranking weight (algorithm fills)
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uniq_feed (user_id, post_id),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE,
  INDEX idx_user_score (user_id, score)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

3.7 Score Me!: `scores`

Per your spec — results are **calculated only on user request and stored**, not computed live.

```
sql

CREATE TABLE scores (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id     BIGINT UNSIGNED NOT NULL,
  target_type  ENUM('job_title','skill','field') NOT NULL,
  target_value VARCHAR(150) NOT NULL,      -- what they scored against
  score_value  FLOAT NOT NULL,             -- the computed result
  breakdown    JSON,                      -- per-factor detail for display
  algo_version VARCHAR(20),               -- so old scores are interpretable
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  INDEX idx_user_target (user_id, target_type, target_value)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Storing `breakdown` (JSON) and `algo_version` means you can show users *why* they got a score and safely change the algorithm later without misrepresenting old results.

3.8 Settings: `user_settings`

Holds toggles like "following enabled." Key-value so new settings need no migration.

```
sql
```

```
CREATE TABLE user_settings (
  user_id      BIGINT UNSIGNED NOT NULL,
  setting_key  VARCHAR(64) NOT NULL,      -- e.g. 'following_enabled'
  setting_value VARCHAR(255) NOT NULL,
  PRIMARY KEY (user_id, setting_key),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

3.9 Email Verification: `email_verifications`

Not needed day one, but the table is ready so adding verification later is drop-in.

```
sql

CREATE TABLE email_verifications (
  id            BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id       BIGINT UNSIGNED NOT NULL,
  token         CHAR(64) NOT NULL UNIQUE,
  expires_at    TIMESTAMP NOT NULL,
  used_at       TIMESTAMP NULL,
  created_at    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

4. Authentication Design

- **Passwords:** `password_hash()` with `PASSWORD_BCRYPT` (or Argon2). Never store plaintext. Verify with `password_verify()`.
- **One account per email:** enforced by `UNIQUE` on `users.email` and `companies.email`. Catch the duplicate-key error and return a friendly message.
- **Sessions:** For the web app, PHP sessions are simplest to start. For the future native app, issue a JWT on login that the app sends in an `Authorization: Bearer` header. Designing the API token-first now means both clients work the same way.
- **Registration flow:** validate input → check email/username uniqueness → hash password → insert `users` row → (later) create `email_verifications` token + send email → return session/token.
- **Verification gate (later):** when enabled, restrict certain actions until `users.is_verified = 1`.
- **Account settings:** every signup field is editable post-signup via `PATCH /api/profile`. Core fields update `users`; flexible fields update `user_profile_attributes`.

5. Feature → Table Mapping

Feature	Primary tables
Register / login (user)	<code>users</code> , <code>email_verifications</code>
Register / login (company)	<code>companies</code>
Profile core (location, pic)	<code>users</code>
Profile extra/new fields	<code>user_profile_attributes</code> , <code>attribute_definitions</code>
Job history	<code>job_history</code>
Degrees / certifications	<code>education</code> , <code>certifications</code>
Skills	<code>skills</code> , <code>user_skills</code>
Career interests	<code>interests</code> , <code>user_interests</code>
Score Me!	<code>scores</code> (reads skills, job_history, education, etc.)
Personal feed page	<code>posts</code>
Main feed	<code>feed_items</code> (+ <code>posts</code> , <code>follows</code>)
Follow users/companies	<code>follows</code>
Settings (e.g. following toggle)	<code>user_settings</code>
Announcement prompts	derived — query profile completeness (no table needed)

Announcement prompts ("finish setting up your profile") don't need storage — compute them on the fly by checking which child tables/attributes are empty for the user.

6. Extensibility & AWS Migration Notes

6.1 Adding new profile fields later

1. **Display-only field?** Insert a row in `attribute_definitions` — done, no migration. Frontend renders it dynamically.

2. **Field you'll score/filter on heavily?** Give it a proper column or child table for performance.

This dual path is the core of keeping the site easy to grow.

6.2 AWS conversion checklist

- ☐ Move MySQL → **RDS (MySQL)**; update credentials in `config.php` only.
- ☐ Move profile pictures `public/uploads/` → **S3**; swap the storage adapter, store S3 URLs in the same `profile_pic`/`logo` columns.
- ☐ Email verification → **SES**.
- ☐ Put app behind HTTPS (ACM cert + load balancer).
- ☐ Externalize secrets to **AWS Secrets Manager** / env vars.
- ☐ Keep `migrations/` so schema changes are repeatable across local → staging → prod.

6.3 Recommended next build steps

1. `config.php` + `Database.php` (PDO connection).
2. `schema.sql` from section 3 — create the DB.
3. Auth endpoints: register + login (users first, companies next).
4. Profile read/update endpoints.
5. Minimal responsive frontend: signup, login, profile page.
6. Layer in follows → posts → feed → Score Me!.

End of v0.1. This foundation supports every feature described and is built to absorb the additions you've said are coming.